

Cahier des Charges - Implémentation Open-Source

Saham Bank Analytics Portal

Auteur : ASSOUMANOU Abdallah

Date : 12 juillet 2026

Contexte : Projet de stage - Transformation du POC en production avec outils open-source

Table des Matières

1. [Présentation du Projet](#)
 2. [Objectifs](#)
 3. [Architecture Technique Open-Source](#)
 4. [Stack Technologique](#)
 5. [Fonctionnalités à Implémenter](#)
 6. [Plan d'Implémentation](#)
 7. [Ressources et Timeline](#)
 8. [Risques et Mitigation](#)
 9. [Livraison et Acceptation](#)
-

1. Présentation du Projet

1.1 Contexte

Le POC Saham Bank Analytics Portal a été développé pour démontrer la faisabilité d'un portail analytique bancaire avec assistant IA (Text-to-SQL). Ce POC utilise actuellement une stack mixte avec des composants propriétaires et open-source.

L'objectif est de transformer ce POC en une solution production-ready utilisant exclusivement des technologies open-source pour :

- Réduire les coûts de licences
- Garantir l'indépendance vis-à-vis des vendors
- Permettre une customisation illimitée
- Assurer la transparence et la sécurité

1.2 Description du POC Actuel

Fonctionnalités existantes :

- Dashboard avec KPIs financiers
- Visualisation interactive (cartes, graphiques)
- Assistant IA (SahamAI) avec Text-to-SQL
- Gestion des rôles (DG, DR, CA, AR, Admin)
- Modules : Pilotage Commercial, Engagements, Qualité Service, Rentabilité, Administration
- Données mockées pour démonstration

Stack actuelle :

- Frontend : React + Vite + TailwindCSS
- Backend : Express.js (Node.js)
- IA : Google Gemini API

- Database : Mock data (JavaScript objects)
- Authentification : Simulation

1.3 Contraintes

Contraintes techniques :

- Utilisation exclusive d'outils open-source
- Compatibilité avec l'existant (migration progressive possible)
- Performance acceptable (< 3s pour requêtes)
- Scalabilité horizontale

Contraintes budgétaires :

- Pas de coûts de licences
- Infrastructure minimale
- Maintenance gérable

Contraintes temporelles :

- Stage de durée limitée
 - Livraison progressive
 - Documentation complète
-

2. Objectifs

2.1 Objectifs Principaux

O1 : Implémenter une stack 100% open-source

- Remplacer Gemini API par LLM open-source
- Migrer vers database open-source
- Utiliser des frameworks open-source

O2 : Assurer la production-readiness

- Architecture scalable
- Gestion des erreurs robuste
- Monitoring et logging
- Tests automatisés

O3 : Maintenir la parité fonctionnelle

- Toutes les fonctionnalités du POC
- Performance équivalente ou supérieure
- UX améliorée

O4 : Documenter pour la maintenance

- Documentation technique
- Guide de déploiement
- Guide de développement

2.2 Objectifs Secondaires

OS1 : Améliorer l'assistant IA

- Meilleure compréhension du langage naturel

- Réduction des hallucinations
- Contexte métier enrichi

OS2 : Optimiser les performances

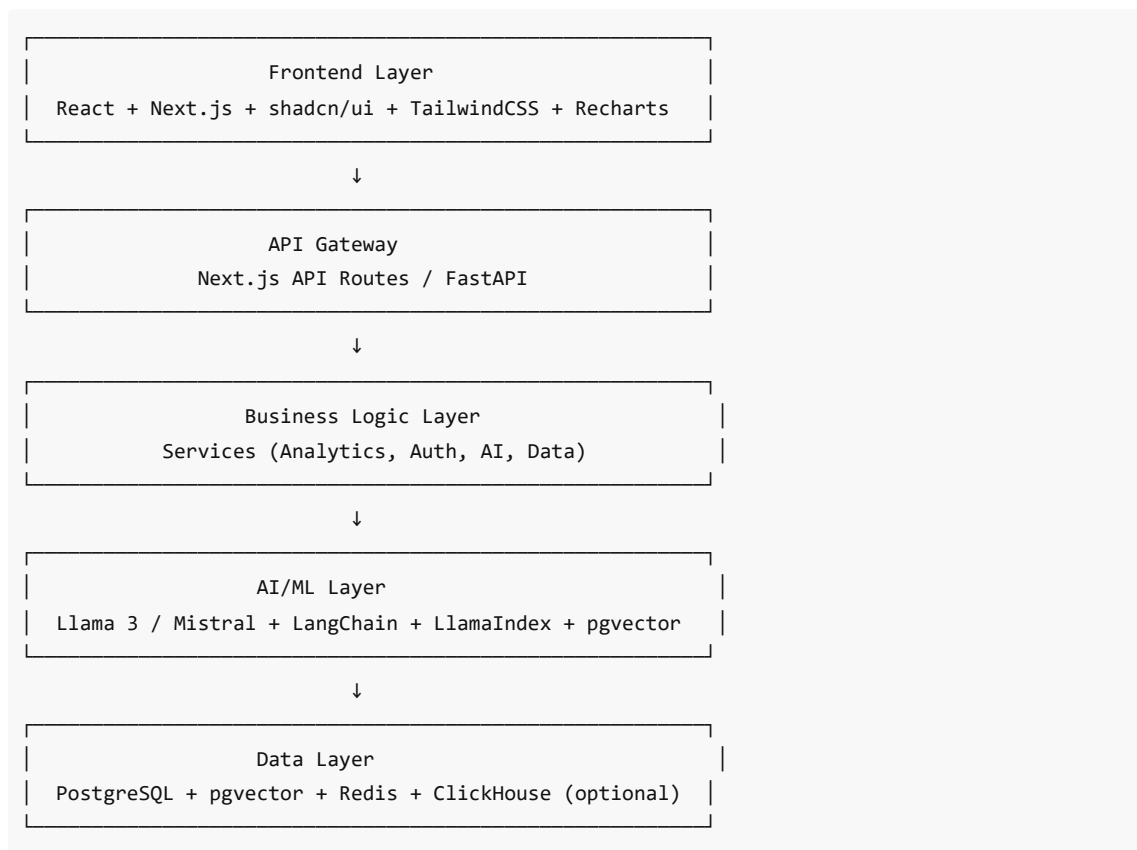
- Caching intelligent
- Optimisation des requêtes
- Lazy loading

OS3 : Préparer l'intégration données réelles

- Connecteurs de données
- ETL pipeline
- Data quality checks

3. Architecture Technique Open-Source

3.1 Architecture Globale



3.2 Architecture Détaillée

3.2.1 Frontend

Framework : Next.js 14 (App Router)

Raisons :

- React-based (compatible avec POC)
- SSR/SSG pour performance

- API routes intégrées
- Excellent SEO
- Communauté massive

UI Library : shadcn/ui + Radix UI + TailwindCSS

Raisons :

- Composants accessibles
- Customisation facile
- Design moderne
- Copy-paste (pas de dépendance lourde)

Visualization : Recharts + Apache ECharts

Raisons :

- Recharts : Simple, React-native
- ECharts : Performance, features riches

State Management : Zustand + React Query

Raisons :

- Zustand : Simple, léger
- React Query : Server state, caching

3.2.2 Backend

Option A : Next.js API Routes (Recommended)

- Full-stack Next.js
- TypeScript natif
- Déploiement simplifié
- Performance optimale

Option B : FastAPI (Python)

- Performance excellente
- Type hints
- Auto-documentation
- Écosystème ML riche

Recommandation : Next.js API Routes pour simplification

3.2.3 Database

Primary Database : PostgreSQL 16

Raisons :

- Relationnel mature
- Extensions (pgvector pour AI)
- Performance excellente
- Open-source robuste
- Compatible avec 多数 ORM

Extension pgvector :

- Vector similarity search

- Support pour RAG
- Intégration native PostgreSQL

Cache : Redis

Raisons :

- Performance caching
- Session storage
- Pub/Sub pour real-time
- Simple à déployer

Optional : ClickHouse

Pour analytics avancés :

- OLAP performance
- Time-series optimisé
- Compression excellente
- SQL compatible

3.2.4 AI/ML Layer

LLM : Llama 3 8B (ou Mistral 7B)

Raisons :

- Open-source
- Performance compétitive
- Commercial use permis
- Self-hosting possible
- Ressources raisonnables (8B)

Alternative : Mistral 7B

Raisons :

- Performance excellente
- French company
- Context window plus grand
- Commercial use permis

Framework RAG : LangChain + LlamaIndex

Raisons :

- LangChain : Framework général, écosystème
- LlamaIndex : Data indexing optimisé
- Complémentaires

Vector Database : pgvector (extension PostgreSQL)

Raisons :

- Intégration native
- Pas de service additionnel
- Performance bonne
- Maintenance simplifiée

Alternative : Qdrant (si performance insuffisante)

3.2.5 Orchestration

Pour data pipelines : Apache Airflow

Raisons :

- Standard de l'industrie
- Communauté massive
- Flexible
- Cloud-native

Alternative : Prefect (plus moderne)

3.2.6 Infrastructure

Containerisation : Docker + Docker Compose

Raisons :

- Standard de l'industrie
- Déploiement simplifié
- Reproductibilité
- Isolation

Reverse Proxy : Nginx

Raisons :

- Performance
- SSL termination
- Load balancing
- Static files

Monitoring : Prometheus + Grafana

Raisons :

- Standard monitoring
- Visualisation riche
- Alerting
- Open-source

4. Stack Technologique

4.1 Frontend Stack

Composant	Technologie	Version	Raison
Framework	Next.js	14+	React-based, SSR, App Router
Language	TypeScript	5+	Type safety
UI Library	shadcn/ui	Latest	Accessible, customisable
Styling	TailwindCSS	3+	Utility-first, performance

State	Zustand	4+	Simple, léger
Server State	React Query	5+	Caching, sync
Visualization	Recharts	2+	React-native
Visualization	Apache ECharts	5+	Performance, features
Forms	React Hook Form	7+	Performance, validation
Validation	Zod	3+	Type-safe validation

4.2 Backend Stack

Composant	Technologie	Version	Raison
Runtime	Node.js	20+	LTS, performance
Framework	Next.js API Routes	14+	Intégré, TypeScript
ORM	Prisma	5+	Type-safe, migrations
Validation	Zod	3+	Type-safe
Auth	NextAuth.js	5+	OAuth, sessions
API Docs	OpenAPI (Swagger)	-	Documentation

4.3 Database Stack

Composant	Technologie	Version	Raison
Database	PostgreSQL	16+	Mature, pgvector
Extension	pgvector	0.5+	Vector search
Cache	Redis	7+	Performance
Optional	ClickHouse	23+	OLAP analytics

4.4 AI/ML Stack

Composant	Technologie	Version	Raison
LLM	Llama 3 8B	Latest	Open-source, performance
Alternative	Mistral 7B	Latest	Performance, French
Framework	LangChain	0.1+	Ecosysteme, chains
Framework	LlamaIndex	0.9+	Data indexing
Vector DB	pgvector	0.5+	Intégré PostgreSQL
Alternative	Qdrant	1.7+	Performance
Inference	llama.cpp / vLLM	Latest	Performance

4.5 DevOps Stack

Composant	Technologie	Version	Raison
Container	Docker	24+	Standard
Orchestration	Docker Compose	2+	Simple
Reverse Proxy	Nginx	1.25+	Performance
Monitoring	Prometheus	2.47+	Standard
Visualization	Grafana	10+	Rich features
Logging	Loki	2.9+	Grafana stack
CI/CD	GitHub Actions	-	Intégré GitHub

5. Fonctionnalités à Implémenter

5.1 Fonctionnalités Core (MVP)

F1 : Authentication & Authorization

Spécifications :

- Login/Logout
- Rôles : DG, DR, CA, AR, Admin
- JWT tokens
- Session management
- Password hashing (bcrypt)

Implementation :

- NextAuth.js
- Prisma User model
- Role-based access control (RBAC)

F2 : Dashboard Principal

Spécifications :

- KPIs financiers (CA, PNB, ROE, etc.)
- Graphiques interactifs
- Filtres temporels
- Drill-down
- Export (CSV, PDF)

Implementation :

- Recharts pour graphiques
- API endpoints pour données
- Caching Redis

F3 : Assistant IA (SahamAI)

Spécifications :

- Text-to-SQL
- Conversation multi-tours
- Contexte métier
- Visualisation des résultats
- Explications naturelles

Implementation :

- Llama 3 8B self-hosted
- LangChain pour orchestration
- pgvector pour RAG
- Fine-tuning sur données bancaires

F4 : Gestion des Rôles

Spécifications :

- Vue adaptée par rôle
- Permissions granulaires
- Admin pour gestion utilisateurs

Implementation :

- RBAC middleware
- Role-based UI rendering
- Admin panel

F5 : Modules Métier

Module 1 : Pilotage Commercial

- KPIs commerciaux
- Segmentation client
- Performance par région

Module 2 : Engagements

- Dossiers de crédit
- Exposition risque
- Suivi portefeuille

Module 3 : Qualité Service

- Réclamations
- NPS
- Temps de réponse

Module 4 : Rentabilité

- PNB par produit
- Commissions
- Coûts

Module 5 : Administration

- Gestion utilisateurs
- Logs d'activité
- Configuration

5.2 Fonctionnalités Avancées (Phase 2)

F6 : Real-time Analytics

Spécifications :

- Mises à jour temps réel
- WebSockets
- Streaming data

Implementation :

- Server-Sent Events (SSE)
- Redis Pub/Sub
- ClickHouse pour OLAP

F7 : Data Connectors

Spécifications :

- Connecteurs multiples (CSV, Excel, API)
- Data ingestion automatisée
- Data quality checks

Implementation :

- Apache Airflow
- Custom connectors
- Great Expectations pour validation

F8 : Advanced AI Features

Spécifications :

- Automated insights
- Anomaly detection
- Recommendations

Implementation :

- scikit-learn pour ML
- Custom models
- Scheduled analysis

6. Plan d'Implémentation

6.1 Phase 1 : Setup et Migration (Semaines 1-2)

Semaine 1 :

- ☐ Initialisation projet Next.js
- ☐ Setup TypeScript + TailwindCSS
- ☐ Setup shadcn/ui
- ☐ Setup Prisma + PostgreSQL
- ☐ Migration structure database
- ☐ Setup Docker Compose

Semaine 2 :

- ☐ Migration frontend (React → Next.js)
- ☐ Setup API routes
- ☐ Migration données mockées → PostgreSQL
- ☐ Setup Redis
- ☐ Tests basiques

Livraison : Projet Next.js fonctionnel avec données en base

6.2 Phase 2 : Core Features (Semaines 3-5)

Semaine 3 :

- ☐ Authentication (NextAuth.js)
- ☐ RBAC implementation
- ☐ Dashboard principal
- ☐ KPIs visualization

Semaine 4 :

- ☐ Assistant IA setup
- ☐ Llama 3 deployment
- ☐ LangChain integration
- ☐ Text-to-SQL basique

Semaine 5 :

- ☐ RAG implementation
- ☐ pgvector setup
- ☐ Contexte métier
- ☐ Tests assistant

Livraison : MVP avec assistant IA fonctionnel

6.3 Phase 3 : Modules Métier (Semaines 6-8)

Semaine 6 :

- ☐ Module Pilotage Commercial
- ☐ Module Engagements
- ☐ Data models

Semaine 7 :

- ☐ Module Qualité Service
- ☐ Module Rentabilité
- ☐ Visualisations avancées

Semaine 8 :

- ☐ Module Administration
- ☐ Gestion utilisateurs

- ☐ Logs et monitoring

** Livraison : ** Tous les modules implémentés

6.4 Phase 4 : Optimisation et Documentation (Semaines 9-10)

Semaine 9 :

- ☐ Performance optimization
- ☐ Caching strategy
- ☐ Error handling
- ☐ Security audit

Semaine 10 :

- ☐ Documentation technique
- ☐ Guide de déploiement
- ☐ Guide de développement
- ☐ Tests finaux

Livraison : Production-ready avec documentation complète

7. Ressources et Timeline

7.1 Ressources Humaines

Rôle : Stagiaire (ASSOUMANOU Abdallah)

Compétences requises :

- React/Next.js
- TypeScript
- PostgreSQL
- Python (pour AI)
- Docker

Compétences à développer :

- LangChain
- LLM deployment
- pgvector
- DevOps basique

7.2 Ressources Techniques

Hardware requis :

Développement :

- CPU : 4 cores minimum
- RAM : 16 GB minimum
- Storage : 50 GB SSD
- GPU : Optionnel (pour LLM fine-tuning)

Production (estimé) :

- CPU : 8 cores
- RAM : 32 GB
- Storage : 200 GB SSD
- GPU : Recommandé pour LLM inference

Logiciels :

- Docker Desktop
- Git
- VS Code
- PostgreSQL client
- Redis client

7.3 Timeline

Total : 10 semaines (2.5 mois)

Phases :

- Phase 1 : 2 semaines (Setup)
- Phase 2 : 3 semaines (Core)
- Phase 3 : 3 semaines (Modules)
- Phase 4 : 2 semaines (Optimisation)

Milestones :

- M1 (Semaine 2) : Setup complet
- M2 (Semaine 5) : MVP fonctionnel
- M3 (Semaine 8) : Modules complets
- M4 (Semaine 10) : Production-ready

7.4 Budget Estimé

Développement :

- Logiciels : 0 € (open-source)
- Hardware : 0 € (machine existante)

Production (estimé mensuel) :

- Cloud server (4 vCPU, 16GB RAM) : 50-100 €/mois
- Storage : 20 €/mois
- Domaine : 10 €/an
- Total : ~70-120 €/mois

Alternative : Self-hosting (coût hardware unique)

8. Risques et Mitigation

8.1 Risques Techniques

R1 : Performance LLM insuffisante

Probabilité : Moyenne

Impact : Élevé

Mitigation :

- Benchmark avant choix
- Alternative : Mistral 7B
- Scaling horizontal
- Caching agressif

R2 : Complexité migration

Probabilité : Élevée

Impact : Moyen

Mitigation :

- Migration progressive
- Tests continus
- Documentation existante
- Backup régulier

R3 : Intégration pgvector problématique

Probabilité : Faible

Impact : Moyen

Mitigation :

- Alternative : Qdrant
- POC préalable
- Documentation pgvector

R4 : Ressources hardware insuffisantes

Probabilité : Moyenne

Impact : Élevé

Mitigation :

- Cloud scaling
- LLM quantization
- Model plus petit (Llama 3 8B)

8.2 Risques Organisationnels

R5 : Délais non respectés

Probabilité : Moyenne

Impact : Moyen

Mitigation :

- Priorisation MVP
- Communication régulière
- Flexibilité scope

R6 : Manque de compétences

Rabilité : Moyenne

Impact : Élevé

Mitigation :

- Formation en amont
- Documentation
- Support communauté
- Mentorat

R7 : Changements requirements

Probabilité : Moyenne

Impact : Moyen

Mitigation :

- Scope clair défini
- Process de change management
- Communication régulière

8.3 Risques Sécurité

R8 : Vulnérabilités sécurité

Probabilité : Faible

Impact : Critique

Mitigation :

- Security audit
- Penetration testing
- Dependencies update régulières
- Best practices (Next.js security)

R9 : Data breaches

Probabilité : Faible

Impact : Critique

Mitigation :

- Encryption at-rest/in-transit
- Access control strict
- Audit trails
- Backup sécurisé

9. Livraison et Acceptation

9.1 Livrables

L1 : Code Source

- Repository GitHub
- Documentation README
- Licences open-source

L2 : Documentation

- Documentation technique
- Guide de déploiement
- Guide de développement

- Architecture diagrams

L3 : Tests

- Tests unitaires
- Tests d'intégration
- Tests E2E
- Rapport de couverture

L4 : Infrastructure

- Docker Compose files
- Configuration Nginx
- Scripts de déploiement
- Monitoring setup

L5 : Formation

- Session de formation équipe
- Vidéos de démonstration
- FAQ

9.2 Critères d'Acceptation

CA1 : Fonctionnalité

- ☐ Toutes les fonctionnalités MVP implémentées
- ☐ Tests passants (80%+ couverture)
- ☐ Pas de bugs critiques

CA2 : Performance

- ☐ Temps de réponse < 3s (95th percentile)
- ☐ LLM inference < 5s
- ☐ Dashboard load < 2s

CA3 : Sécurité

- ☐ Authentification fonctionnelle
- ☐ RBAC correct
- ☐ Pas de vulnérabilités critiques

CA4 : Documentation

- ☐ Documentation complète
- ☐ Guide de déploiement fonctionnel
- ☐ Code commenté

CA5 : Déploiement

- ☐ Déploiement réussi en environnement de test
- ☐ Monitoring fonctionnel
- ☐ Logs accessibles

9.3 Processus d'Acceptation

Étape 1 : Revue Technique

- Revue code avec équipe
- Validation architecture
- Review sécurité

Étape 2 : Tests Utilisateur

- Tests par utilisateurs métier
- Feedback UX
- Validation fonctionnalités

Étape 3 : Déploiement Test

- Déploiement environnement test
- Tests charge
- Validation performance

Étape 4 : Acceptation Formelle

- Signature document d'acceptation
 - Handover documentation
 - Plan de maintenance
-

10. Appendices

10.1 Références

Documentation :

- Next.js Documentation : <https://nextjs.org/docs>
- Prisma Documentation : <https://www.prisma.io/docs>
- LangChain Documentation : <https://python.langchain.com/>
- pgvector Documentation : <https://github.com/pgvector/pgvector>

Tutoriels :

- Next.js Full Stack Course
- LangChain for LLM Apps
- RAG with LlamaIndex

Communautés :

- Next.js Discord
- LangChain Discord
- PostgreSQL Community

10.2 Glossaire

LLM : Large Language Model

RAG : Retrieval-Augmented Generation

RBAC : Role-Based Access Control

SSR : Server-Side Rendering

Ssg : Static Site Generation

OLAP : Online Analytical Processing

ETL : Extract, Transform, Load
MVP : Minimum Viable Product

10.3 Contacts

Stagiaire : ASSOUMANOU Abdallah

Superviseur : [À définir]

Technical Advisor : [À définir]

Document version 1.0

**Dernière mise à jour : 12 juillet 2026

Statut : En attente de validation